Attendify CODE GUIDE

A guided tour of the codebase, written for a student seeing this stack for the first time.

ORIENTATION

CORE MECHANICS

FEATURE WALKTHROUGHS

CRAFT

REFERENCE

Reading Attendify: how a real Expo + Supabase app is put together

This is a full walkthrough of the `attendance` project — a QR-code attendance app for a university, with three roles (Admin, Student, Lecturer). It's written for someone who has done a bit of JavaScript or React but has never seen Expo, Supabase, or a production-shaped mobile codebase before. Every code sample below is pulled straight from the project, not simplified pseudocode.

Expo SDK 54
React Native 0.81, TypeScript

Supabase
Auth, Postgres, Storage, Edge Functions

~40 files
app, components, hooks, services, lib

---

00 How to use this guide

Read modules 01–07 in order first — they explain the ideas every screen in the app leans on (routing, auth, the database, and the "services → hooks → screens" data pattern). Once those click, modules 08–12 walk through each feature top to bottom, and you can jump to whichever one interests you. Modules 13–17 are reference material: come back to them as needed.

> ***Tip*** Keep the actual project open in an editor next to this guide. Every file path here, like `app/(admin)/scanner.tsx`, is real — open it and follow along.

01 What the app does

Attendify replaces paper sign-in sheets. Every student and lecturer has a personal QR code (just their account's unique ID, rendered as a QR image). An admin — or, since a recent feature, a lecturer — opens a camera scanner, points it at that QR code, and the app records "this person was present today" in the database.

There are exactly three roles, and each sees a different set of screens after logging in:

ROLE	WHAT THEY CAN DO
Student	See their own QR code, their attendance history, and their profile. Nothing else — they can't scan anyone.
Lecturer	Everything a student can do, *plus* two extra tabs: scan a student's QR code to mark them present, and see a list of every student they've personally scanned.
Admin	A dashboard with stats, a QR scanner that can scan anyone, full attendance history, lists of every student and lecturer, and the ability to create new accounts and set profile photos.

There's no public sign-up. An admin account is created once by hand (through the Supabase dashboard), and from then on that admin creates every student and lecturer account from inside the app.

02 The tech stack, and why each piece is there

Six libraries do almost all of the work. It's worth understanding what job each one has before reading any screen code.

Expo + Expo Router

Expo is a toolchain on top of React Native that handles the native build, camera access, permissions, and so on, so the app is written almost entirely in JavaScript/TypeScript. **Expo Router** is its navigation library, and it works like Next.js: every file inside `app/` automatically becomes a screen, and the file's path becomes the URL. There's no separate "register this screen" step — see Module 04.

TypeScript

JavaScript with types. Every function's inputs and outputs are declared, so the editor catches a typo like `profile.rol` (should be `role`) before the app ever runs. The shared shapes live in `types/database.ts` (Module 15).

NativeWind

Lets you style React Native components with Tailwind CSS class names — `className="rounded-2xl bg-blue-600 px-4 py-3"` — instead of writing a separate `StyleSheet` object per component. See Module 14.

Supabase

The backend, in one sentence: a hosted Postgres database, a login/authentication system, file storage, and small serverless functions, all reachable from the app through one JavaScript client (`lib/supabase.ts`). No custom backend server was written for this app — Supabase *is* the backend.

TanStack React Query

Manages "server state" — data that lives in the database, not in the app. Instead of writing `useState` + `useEffect` + a loading flag by hand for every screen that needs data, a *hook* like `useStudents()` handles fetching, caching, loading state, and refetching for you. See Module 07.

expo-camera & react-native-qrcode-svg

One library reads QR codes (the scanner), the other draws them (each person's personal pass). See Module 10.

03 Folder tour

The project deliberately separates *what the user sees* from *how data is fetched* from *how data is talked to over the network*. Six folders, each with one job:

FOLDER	JOB
`app/`	Every screen. The file layout *is* the navigation structure (Module 04).
`components/`	Reusable pieces of UI shared across screens — buttons, cards, list rows (Module 13).
`hooks/`	React Query hooks and the auth context. The bridge between screens and services.
`services/`	Plain functions that talk to Supabase directly — no React in this folder at all.
`lib/`	Small framework-agnostic helpers: the Supabase client itself, date formatting, config.
`types/`	Shared TypeScript shapes (`Profile` , `AttendanceRecord` , ...).
`supabase/`	Everything that runs on the server: the database schema (`schema.sql`) and the one Edge Function (Module 12). This is not app code — it's deployed to Supabase separately, not bundled into the app.

Why this matters This layering means a screen component never writes a raw Supabase query itself. It calls a hook; the hook calls a service; the service is the only place that knows the actual table and column names. Change a column name and you fix it in one file, not in every screen that uses it.

04 Routing & navigation: how Expo Router reads the `app/` folder

This is the single most important idea to internalize before anything else makes sense. Here's the actual file tree inside `app/`:

```
app/
├── _layout.tsx              ← runs before every screen (Module 05)
├── index.tsx                 ← the "/" route: decides where to send you
├── login.tsx                 ← the "/login" route
├── onboarding.tsx             ← the "/onboarding" route
├── (member)/                  ← a route GROUP for Student + Lecturer
│   ├── _layout.tsx             ← the bottom tab bar for this group
│   ├── index.tsx                ← Home (QR code)
│   ├── history.tsx               ← Attendance History
│   ├── profile.tsx                ← Profile
│   ├── scan.tsx                    ← Scan (lecturer-only tab)
│   └── my-scans.tsx                 ← My Scans (lecturer-only tab)
└── (admin)/                   ← a route GROUP for Admin
    ├── _layout.tsx             ← the bottom tab bar for this group
    ├── index.tsx                ← Dashboard
    ├── scanner.tsx               ← QR Scanner
    ├── attendance.tsx             ← full Attendance list
    ├── students.tsx                ← Students list
    ├── lecturers.tsx                ← Lecturers list
    ├── create-user.tsx               ← Create Account form
    └── user/[id].tsx                  ← one user's detail screen
```

Three conventions turn this file tree into an app:

- **A file is a screen.** `app/login.tsx` automatically becomes the `/login` route. You never write `Stack.Screen name="login" component={LoginScreen}` anywhere by hand.
- **Parentheses make a "group", not a URL segment.** `(member)` and `(admin)` exist purely to organize files and share a layout — the student's home screen is just `/`, not `/(member)/`.
- **Square brackets are a dynamic segment.** `user/[id].tsx` matches `/user/anything`, and whatever was in that position is readable inside the component via `useLocalSearchParams()`.

How the app decides which screen group you can even see

Both role groups exist in the file tree at all times, but a student should never be able to navigate into `(admin)`. That's enforced in the root layout with Expo Router's `Stack.Protected`:

```
// app/_layout.tsx
<Stack screenOptions={{ headerShown: false }}>
  <Stack.Screen name="index" />
  <Stack.Screen name="onboarding" />

  <Stack.Protected guard={!session}>
    <Stack.Screen name="login" />
  </Stack.Protected>

  <Stack.Protected guard={!!session && profile?.role === "admin"}>
    <Stack.Screen name="(admin)" />
  </Stack.Protected>

  <Stack.Protected guard={!!session && !!profile && profile.role !== "admin"}>
    <Stack.Screen name="(member)" />
  </Stack.Protected>
</Stack>
```

Each `guard` is just a boolean. If it's `false`, that whole screen group doesn't exist in the navigator right now — there's no route to push to it, deep-link to it, or back-button into it. `app/index.tsx` then does the actual routing decision the first time the app opens, using `<Redirect />` to send you to `/login`, `/(admin)`, or `/(member)` based on the same three values.

> ***Beginner tip*** Inside the `(member)` tab bar, the Scan and My Scans tabs use `options={{ href: isLecturer ? undefined : null }}`. Passing `href: null` to a `Tabs.Screen` hides it from the tab bar without deleting the route — that's how one shared layout shows five tabs to a lecturer and three to a student.

05 Authentication & sessions

All login state lives in one place: `hooks/use-auth.tsx`. It exposes an `AuthProvider` (wrapped around the whole app in `app/_layout.tsx`) and a `useAuth()` hook any screen can call to read `session`, `profile`, `isLoading`, `signIn()`, and `signOut()`.

There are two separate pieces of "who is logged in" data, and the distinction matters:

- **Session** — comes straight from Supabase Auth. Proves *who* you are (an email + a token).
- **Profile** — a row from the app's own `profiles` table, fetched separately. Holds the *role* (admin/student/lecturer), name, and avatar — things Supabase Auth doesn't know about.

```
// hooks/use-auth.tsx (simplified)
async function applySession(nextSession) {
  const nextProfile = await resolveProfile(nextSession); // fetch the profiles row
  setSession(nextSession);
  setProfile(nextProfile);
  setIsLoading(false);
}

supabase.auth.onAuthStateChange((_event, nextSession) => {
  applySession(nextSession);
});
```

A real bug, and the fix An earlier version called `setSession(…)` and then separately awaited the profile fetch before calling `setProfile(…)`. That created one render where `session` was set but `profile` was still `null` — a state that matches *none* of the three guards in Module 04, so nothing rendered until the app was force-reloaded. The fix, shown above, is to resolve the profile *first*, then set both pieces of state together in the same tick, so every render is internally consistent. It's a good example of why "loading" is really "these two async things haven't both resolved yet," not just one flag.

06 The database, and Row Level Security from scratch

Two tables. Everything the app does eventually reads or writes one of these:

`PROFILES`	
`id`	same UUID as the Supabase Auth user — this row *is* that person
`full_name`, `email`	display info
`role`	`'admin' | 'student' | 'lecturer'`
`avatar_url`	public URL of their photo, or `null`
`ATTENDANCE`	
`user_id`	who was marked present
`date`, `recorded_at`	when
`recorded_by`	whose account did the scanning
`recorded_by_role`	`'admin' | 'lecturer'` — see the note below

A `unique(user_id, date, recorded_by_role)` constraint on `attendance` is what makes "already recorded today" work — inserting a second row for the same person, date, *and* role fails at the database level, not just in application code. Note the constraint includes the role: an admin's scan and a lecturer's scan of the same student on the same day are deliberately two separate rows, not a duplicate. (The very first version of this constraint was just `unique(user_id, date)`, which meant an admin's scan silently blocked a same-day lecturer scan — a good example of a business rule hiding inside a database constraint.)

What Row Level Security actually is

This is probably the newest idea in this codebase if you've only used simpler backends before. Normally, "can this user read this row?" is a check your *server code* performs — but this app has no custom server; the phone talks to Postgres almost directly through Supabase's API. **Row Level Security (RLS)** moves that permission check into the database itself. Every table has RLS turned on, and every table has *policies* — SQL boolean expressions — that decide, per row, whether the currently logged-in user is allowed to select/insert/update it. If no policy matches, Postgres behaves as if the row doesn't exist. There is no way to bypass this from the app, because the app never gets an unrestricted connection.

```
-- supabase/schema.sql
create policy "attendance_insert" on public.attendance
  for insert
  with check (
    public.is_admin(auth.uid())
    or (
      public.is_lecturer(auth.uid())
      and exists (
        select 1 from public.profiles where id = user_id and role = 'student'
      )
    )
  );
```

Read that as a sentence: "you may insert an attendance row if you're an admin, *or* if you're a lecturer *and* the person you're inserting a row for is a student." A lecturer trying to mark another lecturer present is rejected by Postgres itself — not by a client-side `if` statement that a modified app could skip. (The app *also* checks this client-side, in `components/attendance-scanner.tsx`, purely so the user gets an instant, friendly error instead of waiting for a network round-trip to fail.)

`auth.uid()` is a Postgres function Supabase provides that returns the logged-in user's ID, straight from their auth token. `is_admin(uid)` and `is_lecturer(uid)` are small helper functions defined in the same file, marked `security definer` so they can read the `profiles` table to check someone's role without triggering RLS recursively on `profiles` itself.

> ***Why this matters*** Security lives in exactly one place — the SQL in `supabase/schema.sql` — instead of being scattered across every screen that happens to query a table. Whenever this guide says a screen "fetches X," assume there's an RLS policy quietly deciding what X actually contains for that particular logged-in user.

07 The data pattern: services → hooks → screens

Every piece of data in the app flows through the same three layers. Here's one complete trace, end to end, for "show a student their attendance history":

`history.tsx` → `useAttendanceHistory()` → `getHistory()` → `supabase.from("attendance")`

1. The service — a plain async function, no React involved:

```
// services/attendance.ts
export async function getHistory(userId: string) {
  const { data, error } = await supabase
    .from("attendance")
    .select("*")
    .eq("user_id", userId)
    .order("date", { ascending: false });
  if (error) throw error;
  return data;
}
```

2. The hook — wraps the service in React Query:

```
// hooks/use-attendance.ts
export function useAttendanceHistory(userId: string | undefined) {
  return useQuery({
    queryKey: ["attendance", "history", userId],
    queryFn: () => getHistory(userId as string),
    enabled: !!userId,
  });
}
```

3. The screen — just calls the hook and renders whatever it gets back:

```
// app/(member)/history.tsx
const { profile } = useAuth();
const { data, isLoading, isRefetching, refetch } = useAttendanceHistory(profile?.id);
```

`useQuery` gives the screen `data`, `isLoading` (for a spinner), and `isRefetching` + `refetch` (wired straight into a `RefreshControl` for pull-to-refresh) — all without a single `useState` or `useEffect` written by hand. The `queryKey` array is how React Query caches results and knows when to refetch: any code anywhere that calls `queryClient.invalidateQueries({'{'} queryKey: ["attendance"] {'}'})` — which `hooks/use-attendance.ts`'s `useRecordAttendance` mutation does after every successful scan — marks *every* query whose key starts with `"attendance"` as stale, so History, the admin Attendance list, and My Scans all refetch automatically the moment a new scan happens, with zero manual "tell the other screen to refresh" code.

Query vs. mutation `useQuery` is for *reading* data (runs automatically, can refetch). `useMutation` is for *writing* data (runs only when you call `.mutate()` or `.mutateAsync()`, e.g. on a button press). Look at `hooks/use-attendance.ts`'s `useRecordAttendance` for the mutation side of this same pattern.

08 Walkthrough: the student / lecturer experience

Student and lecturer share the exact same route group, `(member)`, and the same first three screens — because their features genuinely are identical, and duplicating three screens for a cosmetic role difference would just be more code to maintain.

Home — `(member)/index.tsx`

Shows the signed-in person's name, role badge, and their QR code. The QR code's content is nothing more than their own `profile.id` (a UUID) — `components/qr-code-card.tsx` just hands that string to the `react-native-qrcode-svg` component:

```
<QRCode value={value} size={200} color="#0f172a" backgroundColor="#ffffff" />
```

Whoever scans it later just needs to look that same UUID up in `profiles`.

History — `(member)/history.tsx`

Every attendance row for this person, newest first, each showing a "Marked by" badge for *admin* or *lecturer* (reading the `recorded_by_role` column from Module 06) so a student can tell the two kinds of scan apart.

Profile — `(member)/profile.tsx`

Read-only account info and a Sign Out button. Students and lecturers can't change their own photo — only an admin can (Module 11).

Lecturer-only: Scan & My Scans

`(member)/scan.tsx` renders the exact same `components/attendance-scanner.tsx` the admin uses (Module 10), just with `allowedRoles={'{["student"]}'}` so a lecturer can only scan students. `(member)/my-scans.tsx` lists everyone *that specific lecturer* has personally recorded, by querying attendance rows `where recorded_by = <their own id>`.

09 Walkthrough: the admin experience

Dashboard — `(admin)/index.tsx`

Three stat cards (present today, total students, total lecturers) built from three separate hooks (`useTodayCount` , `useStudents` , `useLecturers`) pulled together in one screen, plus quick-action buttons to Scan, view Attendance, or Add Account.

Students / Lecturers lists

`(admin)/students.tsx` and `lecturers.tsx` are almost identical files — a `FlatList` over `useStudents()` / `useLecturers()` , rendering a `UserListItem` per row. Tapping a row navigates to `(admin)/user/[id].tsx` , that person's detail screen, where the admin can change their photo.

Create Account — `(admin)/create-user.tsx`

A form: role toggle (student/lecturer), name, email, and an auto-generated temporary password (`lib/password.ts` has a `generateTempPassword()` helper — with a "Regenerate" button next to it). Submitting doesn't talk to Supabase's database directly; it calls the Edge Function covered in Module 12, because creating another person's login requires a permission level the phone app is never allowed to hold.

10 QR scanning, end to end

Both the admin scanner and the lecturer scanner render the same component, `components/attendance-scanner.tsx` — written once, used in two places, with one prop (`allowedRoles`) changing its behavior. Here's the full sequence that runs every time a QR code is successfully read:

`camera reads QR text` → `look up that UUID in profiles` → `check role is allowed` → `insert into attendance` → `toast or alert`

```
// components/attendance-scanner.tsx (trimmed)
const scannedProfile = await getProfileById(data.trim());
if (!scannedProfile) { Alert.alert("User not found", …); return; }

if (allowedRoles && !allowedRoles.includes(scannedProfile.role)) {
  Alert.alert("Not allowed", …); return;
}

const result = await recordAttendance.mutateAsync({
  userId: scannedProfile.id,
  recordedBy: profile.id,
  recordedByRole: profile.role,
});

if (result.status === "already_recorded") {
  Alert.alert("Already recorded", …);
} else {
  showSuccessToast(`${scannedProfile.full_name} marked present`);
}
```

A `scanned` boolean lock prevents the camera from firing this whole sequence again while one scan is still being processed, or while a success toast is showing — it resets automatically after the toast disappears (`components/ui/toast.tsx`), so an admin scanning a queue of students never has to tap anything between scans, only when something needs their attention (a duplicate or an unknown code).

`recordAttendance` itself (in `services/attendance.ts`) first checks whether this exact person/date/role combination already has a row, and if the insert still fails with Postgres error code `23505` (a unique constraint violation — two scans racing each other at almost the same instant), it's treated as "already recorded" rather than a crash.

11 Profile pictures

Photos are admin-managed only — a student or lecturer sees their photo but can't change it. Three pieces make this work:

- `components/ui/avatar.tsx` — a small component that shows the photo if `avatar_url` is set, or the person's initial in a colored circle if not. Used everywhere a person appears in the UI.
- `components/avatar-picker.tsx` — an `Avatar` wrapped in a `Pressable` with a camera badge; tapping it opens the photo library via `expo-image-picker`.
- `services/storage.ts` — does the actual upload:

```
// services/storage.ts
const file = new File(localUri);
const arrayBuffer = await file.arrayBuffer();
const path = `${'{'}userId{'}'}/${'{'}Date.now(){'}'}.${'{'}extension{'}'}`;

await supabase.storage.from("avatars").upload(path, arrayBuffer, {
  contentType, upsert: true,
});
```

Two details worth noticing: the filename includes `Date.now()`, so replacing a photo always gets a brand-new URL — without that, a phone or browser could keep showing a cached copy of the old image at the same URL. And uploads only succeed for an admin, because the `avatars` Storage bucket has its own RLS-style policies (Storage in Supabase is backed by Postgres too, so the same permission model from Module 06 applies to files, not just table rows).

12 The one Edge Function, and why it has to exist

Creating a new login (an email + password pair) requires Supabase's `service_role` key — a key that can bypass *every* RLS policy in the entire database. That key must never exist inside a mobile app, because anyone could extract it from the compiled app and gain full access to every table. So account creation can't happen directly from the phone.

The fix is a tiny server-side program — a Supabase **Edge Function** — that holds that dangerous key instead. It lives at `supabase/functions/create-user/index.ts`, runs on Supabase's servers (not in the app bundle), and is deployed separately via `supabase functions deploy create-user`. The phone calls it like a small API endpoint:

```
// services/admin.ts
const { data, error } = await supabase.functions.invoke("create-user", {
  body: { email, password, fullName, role },
});
```

Inside the function, before it does anything privileged, it re-checks who's calling:

```
// supabase/functions/create-user/index.ts (trimmed)
const callerClient = createClient(supabaseUrl, anonKey, {
  global: { headers: { Authorization: authHeader } },
});
const { data: { user: caller } } = await callerClient.auth.getUser();

const { data: callerProfile } = await callerClient
  .from("profiles").select("role").eq("id", caller.id).single();

if (callerProfile?.role !== "admin") {
  return json({ error: "Only admins can create accounts" }, 403);
}

// only past this point does it switch to the privileged client:
const adminClient = createClient(supabaseUrl, serviceRoleKey);
await adminClient.auth.admin.createUser({ email, password, ... });
```

Why this matters Notice the function uses *two* different Supabase clients: an ordinary one (with the caller's own login token) just to check "is this really an admin?", and only then a second, privileged one to actually create the account. The dangerous key is never used until the caller has already proven who they are, using the same RLS-respecting checks as everywhere else in the app.

13 The component library

Small, single-purpose pieces in `components/ui/`, reused across nearly every screen:

COMPONENT	PURPOSE
`Button`	Four variants (primary/secondary/danger/outline), optional icon, a built-in loading spinner state, and a subtle press-scale animation via `react-native-reanimated`.
`Card`	The rounded, shadowed container almost every block of content sits inside.
`Badge` (`RoleBadge`)	A colored pill for admin/student/lecturer — purple, blue, green respectively.
`Avatar`	Photo or initials fallback (Module 11).
`Toast`	The green success banner used after a scan (Module 10).
`EmptyState`	The icon + message shown when a list has zero items — "No students yet," etc.
`LoadingSpinner`	A centered spinner, shown while a query's `isLoading` is true.

One level up, `components/` also holds slightly bigger, feature-specific pieces that still get reused: `AttendanceListItem` and `UserListItem` (one row in a list), `QrCodeCard`, and `AttendanceScanner` (Module 10).

14 Styling with NativeWind

Ordinary React Native styling means a `StyleSheet.create({'{'} … {'}'})` object per component. NativeWind lets you write Tailwind's utility classes directly on the `className` prop instead, and it compiles them to real native styles at build time — there's no CSS engine running on the phone.

```
<View className="flex-row items-center gap-3 rounded-2xl bg-white p-4 shadow-sm dark:bg-slat
```

Reading that: a horizontal flex row, centered items, a gap between children, big rounded corners, white background, padding, a soft shadow — and `dark:bg-slate-800` swaps the background automatically when the phone is in dark mode, with zero extra code. NativeWind reads the OS color scheme through React Native's `Appearance` API for you.

The three config files that make this work — `babel.config.js`, `metro.config.js`, and `tailwind.config.js` — are boilerplate you set up once and rarely touch again.

15 TypeScript types

Every shape the app passes around is defined once, in `types/database.ts`, and imported everywhere else:

```
export type Role = "admin" | "student" | "lecturer";

export type Profile = {
  id: string;
  full_name: string;
  email: string;
  role: Role;
  avatar_url: string | null;
  created_at: string;
};

export type AttendanceRecord = {
  id: string;
  user_id: string;
  date: string;
  recorded_at: string;
  recorded_by: string | null;
  recorded_by_role: Extract<Role, "admin" | "lecturer"> | null;
};
```

`Extract<Role, "admin" | "lecturer">` is TypeScript's way of saying "the subset of `Role` that is either `'admin'` or `'lecturer'`" — it exists because a *student* can never be the one who recorded an attendance row, so that narrower type is more accurate than reusing the full `Role` type here. There's also an `AttendanceWithProfile` type, used whenever a query joins an attendance row together with the person it belongs to (the admin's Attendance list, a lecturer's My Scans).

Why this matters Because these types are shared, a service function's return type and the screen that consumes it are checked against the *same* definition. If a column gets renamed in `types/database.ts`, every file that used the old name turns into a compile error immediately — not a runtime crash discovered later on someone's phone.

16 Glossary

TERM	MEANING HERE
JSX	Writing HTML-like markup inside JavaScript, e.g. `<Text>hi</Text>`. React Native uses it with native components (`View`, `Text`) instead of web ones (`div`, `span`).
Hook	A function starting with `use` that can hold state or side effects inside a component, e.g. `useState`, `useAuth()`, `useStudents()`.
Component	A reusable function that returns JSX — the building block of the whole UI.
Props	The inputs passed into a component, e.g. `<Avatar name={'{'}profile.full_name{'}'} />`.
State	Data a component remembers between renders and can update, e.g. what's typed into a text field.
RLS	Row Level Security — Postgres deciding, per row, who can read/write it. See Module 06.
RPC / Edge Function	Code that runs on Supabase's servers rather than the phone. See Module 12.
JWT	JSON Web Token — the signed, tamper-proof token Supabase Auth issues on login, proving who you are on every later request.
Mutation	A React Query operation that changes data (insert/update/delete), as opposed to a query (read-only).
`useState` vs. React Query	`useState` is for data that only ever lives in this component (a form field). React Query is for data that lives in the database and might be shared/stale/refetched.

17 Exercises to try

The best way to learn a codebase is to change something small in it and watch what breaks. In rough order of difficulty:

1. Add a fourth stat card to the admin Dashboard — e.g. "Present this week." You'll write a new function in `services/attendance.ts`, a new hook in `hooks/use-attendance.ts`, and one new `<StatCard>` in `app/(admin)/index.tsx`. This exercises the full services → hooks → screens pattern from Module 07.
2. Add a "Copy user ID" button to the Profile screen, using React Native's `Clipboard` API. Pure UI, no data layer needed.
3. Let a student see *who* recorded their attendance by name, not just their role. You'll need to extend the RLS policy on `profiles` so a student is allowed to read the profile of whoever scanned them — a good exercise in reasoning about Module 06's security model, since it's a real permission you have to grant carefully.
4. Add a fourth role, "TA" (teaching assistant), with the same permissions as a lecturer. Trace every file that mentions `"lecturer"` — the `Role` type, the RLS policies, the tab bar guard, the scanner's `allowedRoles` — and you'll have touched almost every module in this guide.

That's the whole app. If something here didn't fully click, it's almost always faster to open the real file and read it next to this explanation than to re-read the prose alone — every path in this guide is real, and the project is small enough to read end to end in an afternoon.